Jodie Butterworth

Sprint2 CPT_S 451

Implement your database using SQL.

I used the following code in a Oracle:

```
CREATE TABLE PROJECTCAMPAIGNREV
(
campaign_id NUMBER,
start_date DATE NOT NULL,
end_date DATE NOT NULL,
funding_goal DECIMAL(19,4) NOT NULL,
CONSTRAINT campaign_pk_rev PRIMARY KEY (campaign_id)
);

CREATE TABLE PROJECTCAMPAIGNTITLEREV
(
campaign_id NUMBER,
title VARCHAR2(255),
description VARCHAR2(255) NOT NULL,
```

```
CONSTRAINT fk_campaigntitle_rev FOREIGN KEY (campaign_id) REFERENCES
ProjectCampaignRev(campaign_id),

CONSTRAINT pk_camp_title_rev PRIMARY KEY (campaign_id, title)

);



CREATE TABLE PROJECTUSERREV

(

user_id NUMBER,

first_name VARCHAR2(255) NOT NULL,

last_name VARCHAR2(255) NOT NULL,

password_hash VARCHAR2(255) NOT NULL,

role VARCHAR2(20) CHECK(role IN('user','admin'))

CONSTRAINT user_pk_rev PRIMARY KEY (user_id)

);



CREATE TABLE PROJECTTRANSACTIONREV

(

transaction_id NUMBER,

campaign_id NUMBER,

user_id NUMBER,

authorization_num VARCHAR2(255) NOT NULL,

payment_type VARCHAR2(255) NOT NULL,
```

```
payment_date DATE NOT NULL,

amount DECIMAL(19,4) NOT NULL, CHECK(amount>0),

CONSTRAINT fk_campaigntrans_rev FOREIGN KEY (campaign_id) REFERENCES
ProjectCampaignRev(campaign_id),

CONSTRAINT fk_user_rev FOREIGN KEY (user_id) REFERENCES ProjectUserRev(user_id),

CONSTRAINT pk_transaction_rev PRIMARY KEY (transaction_id)

);



CREATE TABLE PROJECTEMAILREV

(

user_id NUMBER,

email VARCHAR2(255) NOT NULL,

CONSTRAINT fk_useremail_rev FOREIGN KEY (user_id) REFERENCES ProjectUserRev(user_id),

CONSTRAINT pk_email_rev PRIMARY KEY (user_id, email)

);



CREATE TABLE PROJECTSOCIALMEDIAREV

(

user_id NUMBER,

socialmedia VARCHAR2(255) NOT NULL,

CONSTRAINT fk_usersocialmedia_rev FOREIGN KEY (user_id) REFERENCES
ProjectUserRev(user_id),
```

```
CONSTRAINT pk_socialmedia_rev PRIMARY KEY (user_id, socialmedia)

);



CREATE TABLE PROJECTUSERCAMPAIGNREV

(

campaign_id NUMBER,

user_id NUMBER,

CONSTRAINT fk_campaignuser_rev FOREIGN KEY (campaign_id) REFERENCES
ProjectCampaignRev(campaign_id),

CONSTRAINT fk_usercampaign_rev FOREIGN KEY (user_id) REFERENCES
ProjectUserRev(user_id),

CONSTRAINT pk_user_campaign_rev PRIMARY KEY (campaign_id, user_id)

);
```

Business Rules Enforcement

Business rules as described on previous assignment:

1. Two users could have the same first and last names. This duplication is handled by giving each user a different user_id, so duplicate names will not cause problems.

2. A user could have multiple email addresses. This is handled by creating a separate table for the emails. Storing multiple email addresses in the same field is difficult to retrieve and edit. Storing the emails in a table where each one is connected to the user_id makes it easier to find and edit a user's emails.

3. If the campaign start date and end date are not set and displayed, users will not know when the campaign is running. The start and end date also set the time limits for collecting the transactions. Therefore, the 'start_date' and 'end_date' are set to NOT NULL so the users and the admins know when the campaign will be running.

4. In the campaign, the funding goal lets people know how much the campaign is trying to raise. Users would not know how much is needed in donations without this field, and admins would not know if the campaign is successful without a goal. So, the 'funding_goal' field is set to NOT NULL to ensure that everyone knows the campaign goal

5. In the campaign, the description tells the user what the campaign is for and why they should donate. The 'description' field is set to NOT NULL to make sure that every campaign is explained and easily identified.

6. In the user, the first name and last name are needed to identify and contact each user. The 'first_name' and 'last_name' are set to NOT NULL so that all the user names are recorded and connected to the user donations.

7. In the user, the password hash is used to log the user in. For access to the account to be secure, a password must be recorded and entered. For the password to be secure, it must be encrypted. The 'password_hash' field is set to NOT NULL to ensure that the account is secured.

8. In the user, the type of user is recorded to determine if they are a user or an admin. The user and admin get access to different features, and the view changes based on the person's role. The 'role' field has CHECK(role IN('user','admin')) to verify that user as one of these two types of users so that the program will allow access to the correct features.

Edge Cases & Failure Handling

Edge Cases as identified on previous assignment:

1. Two users could have the same first and last names. This duplication is handled by giving each user a different user_id, so duplicate names will not cause problems.

2. A user could have multiple email addresses. This is handled by creating a separate table for the emails. Storing multiple email addresses in the same field is difficult to retrieve and edit. Storing the emails in a table where each one is connected to the user_id makes it easier to find and edit a user's emails.

3. A user could also have multiple social media accounts. Again, a separate table is made to handle the multiple entries per user. Storing the social media data in a separate table with a user_id FK makes it easier to find and edit a user's social media data.

4. Even though transactions would not exist without a user donating to the campaign, we want to keep financial records of the transactions after a user quits the site or the campaign is ended. Therefore, old user and campaign data is not deleted because the child table PROJECTTRANSACTION still needs it for financial records. RESTRICT is used when deleting is necessary for a row in campaign or user.

5. The PROJECTUSER and PROJECTCAMPAIGN tables are in a M : M relationship because one user can donate to many campaigns and one campaign can have donations from multiple users. A separate table with FK for both user_id and campaign_id is used to connect all the users to their campaigns and vice versa. The PK for the table is user_id and campaign_id together. This makes it easier correctly assign all the campaigns that a user has donated to and all the users that have given to a campaign.

6. The attribute for funding_received is calculated from all the transactions for that campaign. This could result in incorrect results if the wrong transactions are used. This is solved by querying the transactions with the campaign_id FK and using sum on them. This way only transactions from the right campaign will be added.

7. The attribute for title under campaign is useful for identifying a campaign, but depending on the logos or information in the description, it may be unnecessary or redundant. Therefore, it is possible to leave the title null in cases where it is not wanted.

Testing of Database with Normalization

Testing with queries, as shown on previous assignment:

(Oracle recommended some of the templates and I changed them, that might be AI in FreeSQL)

Oracle recommended:

```
INSERT INTO PROJECTCAMPAIGNREV (
    CAMPAIGN_ID,
    START_DATE,
    END_DATE,
    FUNDING_GOAL
) VALUES ( 1,
        TO_DATE('15-03-2026', 'DD-MM-YYYY'),
        TO_DATE('16-03-2026', 'DD-MM-YYYY'),
        100.01
        );

INSERT INTO PROJECTCAMPAIGNTITLEREV (
    CAMPAIGN_ID,
    TITLE,
    DESCRIPTION
) VALUES ( 1,
        'campaigntest1',
        'A test record for campaign table');

INSERT INTO PROJECTUSERREV (
    USER_ID,
    FIRST_NAME,
    LAST_NAME,
```

```
    PASSWORD_HASH,
    ROLE
) VALUES ( 1,
        'Test',
        'User',
        'password',
        'user' );

INSERT INTO PROJECTTRANSACTIONREV (
    TRANSACTION_ID,
    CAMPAIGN_ID,
    USER_ID,
    AUTHORIZATION_NUM,
    PAYMENT_TYPE,
    PAYMENT_DATE,
    AMOUNT
) VALUES ( 1,
        1,
        1,
        'T001',
        'credit_card',
        TO_DATE('15-03-2026', 'DD-MM-YYYY'),
        10.01 );
```

Display whole table:

```
SELECT
*
FROM
    PROJECTCAMPAIGNREV;

SELECT
*
FROM
    PROJECTUSERREV;

SELECT
*
FROM
    PROJECTTRANSACTIONREV;
```

Create a new user:

```
INSERT INTO PROJECTUSERREV (
    USER_ID,
    FIRST_NAME,
    LAST_NAME,
    PASSWORD_HASH,
```

```
    ROLE
) VALUES ( 5,
        'Bill',
        'Purrin',
        'password123',
        'user' );

INSERT INTO PROJECTEMAILREV (
    USER_ID,
    EMAIL
) VALUES ( 5,
        'billyp1@gmail.com' );

INSERT INTO PROJECTEMAILREV (
    USER_ID,
    EMAIL
) VALUES ( 5,
        'billygoat1@gmail.com' );

INSERT INTO PROJECTSOCIALMEDIAREV (
    USER_ID,
    SOCIALMEDIA
) VALUES ( 5,
        'billybilly@facebook.com' );
```

Admin creates a new campaign:

```
INSERT INTO PROJECTCAMPAIGNREV (
    CAMPAIGN_ID,
    START_DATE,
    END_DATE,
    FUNDING_GOAL,
    FUNDING_RECEIVED,
) VALUES ( 3,
        TO_DATE('31-03-2026', 'DD-MM-YYYY'),
        TO_DATE('16-06-2026', 'DD-MM-YYYY'),
        100.01,
        1.01;

INSERT INTO PROJECTCAMPAIGNTITLEREV (
    CAMPAIGN_ID,
    TITLE,
    DESCRIPTION
) VALUES ( 3,
        'Better Living through Tacos',
        'The sequel to my self-help book Insights on Extra Cheese. Better Living through Tacos explores the fast-food versus fine-dining taco in us all. I request donations from the small but dedicated food-based self-help book lovers.');
```

User makes a new donation for a campaign:

```
INSERT INTO PROJECTTRANSACTIONREV (
    TRANSACTION_ID,
    CAMPAIGN_ID,
    USER_ID,
    AUTHORIZATION_NUM,
    PAYMENT_TYPE,
    PAYMENT_DATE,
    AMOUNT
) VALUES ( 4,
        3,
        5,
        'TACO4',
        'billy_credit_card',
        TO_DATE('31-03-2026', 'DD-MM-YYYY'),
        5.01 );
```

Admin removing an unneeded column from PROJECTCAMPAIGN:

```
ALTER TABLE PROJECTCAMPAIGNREV
DROP COLUMN funding_received;
```

User changes email:

UPDATE PROJECTEMAILREV

SET email = 'billynewemail@gmail.com'

WHERE email = 'billyp1@gmail.com';

Admin changes campaign end-date:

UPDATE PROJECTCAMPAIGNREV

SET end_date = TO_DATE('30-06-2026', 'DD-MM-YYYY')

WHERE campaign_id = 3;

Admin attempts to delete a user: (Note: this fails, and we want it to)

DELETE FROM PROJECTUSERREV

WHERE user_id = 5;

Admin attempts to delete a campaign: (Note: this fails, and we want it to)

DELETE FROM PROJECTCAMPAIGNREV

WHERE campaign_id = 3;

Admin queries the full name of the users:

SELECT first_name || ' ' || last_name AS full_name

```
FROM PROJECTUSERREV;
```

Admin queries the total donations for a campaign:

```
SELECT sum(amount) total_donations
FROM PROJECTTRANSACTIONREV
WHERE campaign_id = 3;
```

User asks to list their donation amounts and dates

```
SELECT amount, payment_date
FROM PROJECTTRANSACTIONREV
WHERE user_id = 5;
```

Insert connecting values into PROJECTUSERCAMPAIGN:

```
INSERT INTO PROJECTUSERCAMPAIGNREV (
    CAMPAIGN_ID,
    USER_ID
) VALUES ( 3,
          5);
```

Display user first and last name with the amount donated by them

SELECT first_name, last_name, amount FROM PROJECTUSERREV, PROJECTTRANSACTIONREV

WHERE PROJECTUSERREV.user_id = PROJECTTRANSACTIONREV.user_id;

Display transaction amount with payment date and user last name

SELECT amount, payment_date, last_name FROM PROJECTTRANSACTIONREV, PROJECTUSERREV

WHERE PROJECTUSERREV.user_id = PROJECTTRANSACTIONREV.user_id;

Display the user first and last name with the corresponding emails.

SELECT first_name, last_name, email FROM PROJECTUSERREV, PROJECTEMAILREV

WHERE PROJECTUSERREV.user_id = PROJECTEMAILREV.user_id;

Display the user first and last name with the corresponding social medias.

SELECT first_name, last_name, socialmedia FROM PROJECTUSERREV, PROJECTSOCIALMEDIAREV

WHERE PROJECTUSERREV.user_id = PROJECTSOCIALMEDIAREV.user_id;

Display users who donated amounts greater than 20 dollars

SELECT first_name, last_name, amount FROM PROJECTUSERREV, PROJECTTRANSACTIONREV

WHERE PROJECTUSERREV.user_id = PROJECTTRANSACTIONREV.user_id

AND PROJECTTRANSACTIONREV.amount > 20;

Build a **basic UI (web-based preferred)** connected to your database.

I built a UI, but it took a while use troubleshoot it into connecting. I used ChatGPT, though it did badly at it. It created html and app.py and then repetitively made the same mistakes for several hours in troubleshooting.

This is the app.py:

```
from flask import Flask, render_template, request, redirect, url_for, session

import oracledb

from werkzeug.security import generate_password_hash, check_password_hash


app = Flask(__name__)

app.secret_key = "key"


# ============================================================

# ORACLE THIN CONNECTION

# ============================================================

def get_connection():

    return oracledb.connect(

        user="system",

        password="pasword",

        dsn="localhost:port/name"

    )


# ============================================================

# HOME ROUTE

# ============================================================
```

```
@app.route('/')
def home():
    return redirect(url_for('login'))

# ============================================================
# LOGIN
# ============================================================
@app.route('/login', methods=['GET', 'POST'])
def login():

    if request.method == 'POST':

        username = request.form.get('username')
        password = request.form.get('password')

        conn = get_connection()

        try:
            with conn.cursor() as cursor:

                cursor.execute("""
                    SELECT user_id, first_name, last_name, password_hash, role
                    FROM PROJECTUSERREV
                    WHERE first_name = :first_name
```

```
            """, {"first_name": username})

            user = cursor.fetchone()

            if not user:
                return render_template("login.html", error="User not found")

            user_id, first_name, last_name, password_hash, role = user

            if not check_password_hash(password_hash, password):
                return render_template("login.html", error="Invalid password")

            session['user_id'] = user_id
            session['role'] = role

            if role == "admin":
                return redirect(url_for("admin_dashboard"))
            else:
                return redirect(url_for("user_dashboard"))

        finally:
            conn.close()

    return render_template("login.html")
```

```
# ============================================================
# USER DASHBOARD
# ============================================================
@app.route('/user')
def user_dashboard():

    if 'user_id' not in session:
        return redirect(url_for('login'))

    conn = get_connection()

    try:
        with conn.cursor() as cursor:

            # USER INFO
            cursor.execute("""
                SELECT first_name, last_name
                FROM PROJECTUSERREV
                WHERE user_id = :user_id
            """, {"user_id": session['user_id']})

            user = cursor.fetchone()
```

```
            if not user:
                return "User not found"

            # EMAILS
            cursor.execute("""
                SELECT email
                FROM PROJECTEMAILREV
                WHERE user_id = :user_id
            """, {"user_id": session['user_id']})

            emails = [row[0] for row in cursor.fetchall()]

        return render_template(
            "user_dashboard.html",
            first_name=user[0],
            last_name=user[1],
            emails=emails
        )

    finally:
        conn.close()

# ============================================================
# ADMIN DASHBOARD
```

```
# ============================================================

@app.route('/admin')

def admin_dashboard():


    if 'user_id' not in session or session.get('role') != 'admin':

        return redirect(url_for('login'))


    return render_template("admin_dashboard.html")


# ============================================================

# CREATE USER

# ============================================================

@app.route('/create_user', methods=['GET', 'POST'])

def create_user():


    if request.method == 'POST':


        first_name = request.form.get('first_name')

        last_name = request.form.get('last_name')

        password = request.form.get('password')


        emails = [e for e in request.form.getlist('emails[]') if e.strip()]

        socials = [s for s in request.form.getlist('socials[]') if s.strip()]
```

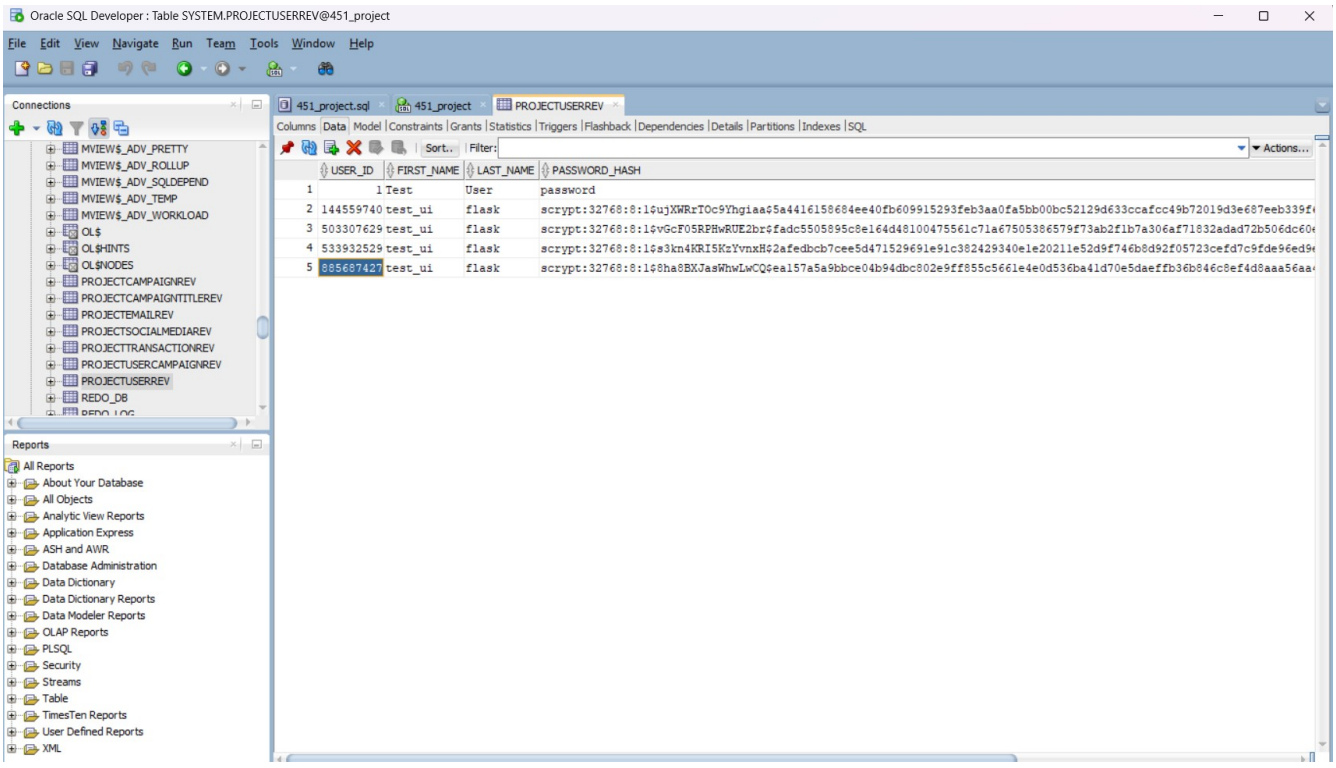
```
if not first_name or not last_name or not password:
    return render_template("create_user.html", error="Missing required fields")

if len(emails) == 0:
    return render_template("create_user.html", error="At least one email required")

hashed_password = generate_password_hash(password)

# SAFE UNIQUE ID GENERATION
user_id = abs(hash(first_name + emails[0])) % 1000000000

conn = get_connection()

try:
    with conn.cursor() as cursor:

        # INSERT USER
        cursor.execute("""
            INSERT INTO PROJECTUSERREV
            (user_id, first_name, last_name, password_hash, role)
            VALUES (:user_id, :first_name, :last_name, :password_hash, 'user')
        """, {
            "user_id": user_id,
            "first_name": first_name,
```

```
    "last_name": last_name,
    "password_hash": hashed_password
})

# INSERT EMAILS
for email in emails:
    cursor.execute("""
        INSERT INTO PROJECTEMAILREV
        (user_id, email)
        VALUES (:user_id, :email)
    """, {
        "user_id": user_id,
        "email": email
    })

# INSERT SOCIAL MEDIA
for social in socials:
    cursor.execute("""
        INSERT INTO PROJECTSOCIALMEDIAREV
        (user_id, socialmedia)
        VALUES (:user_id, :socialmedia)
    """, {
        "user_id": user_id,
        "socialmedia": social
```

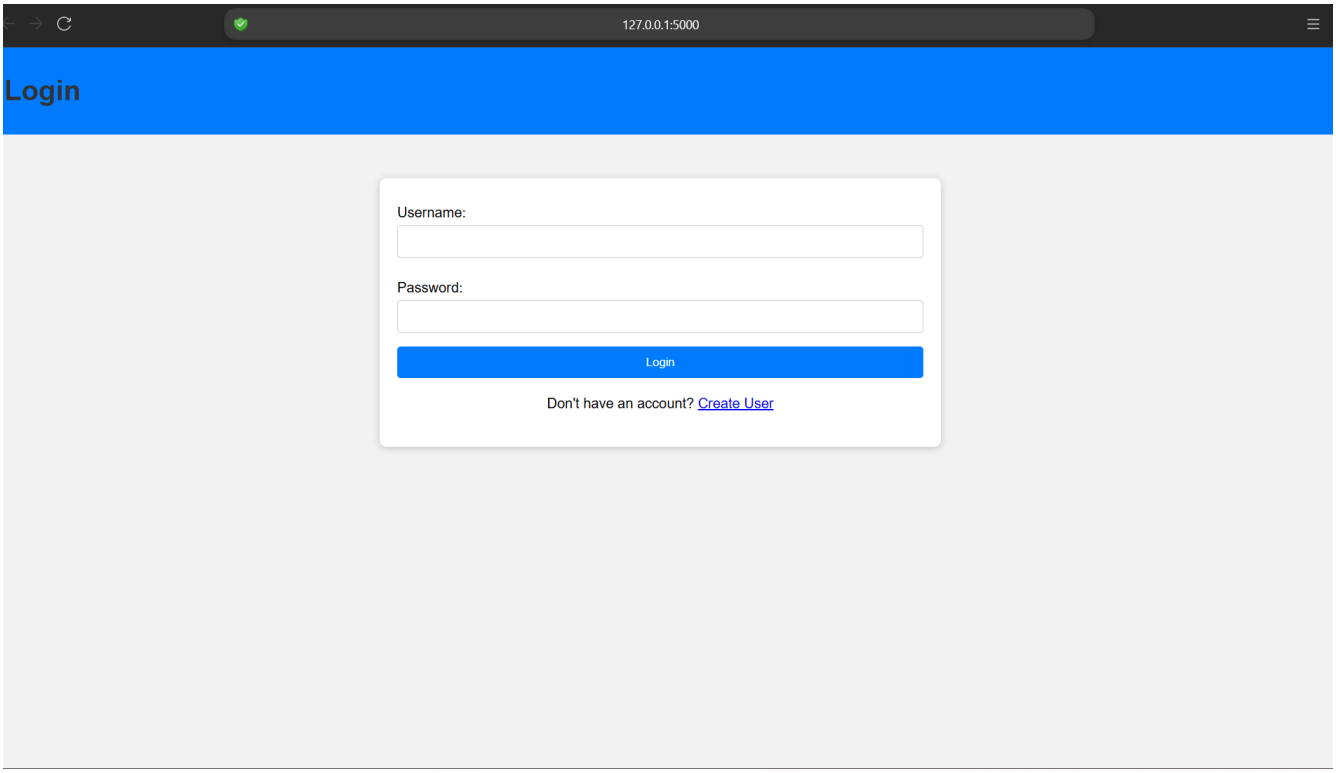
```
                })

            conn.commit()

            session['user_id'] = user_id
            session['role'] = 'user'

            return redirect(url_for('user_dashboard'))

        except Exception as e:
            conn.rollback()
            return render_template("create_user.html", error=str(e))

        finally:
            conn.close()

    return render_template("create_user.html")

# ============================================================
# LOGOUT
# ============================================================
@app.route('/logout')
def logout():
    session.clear()
```

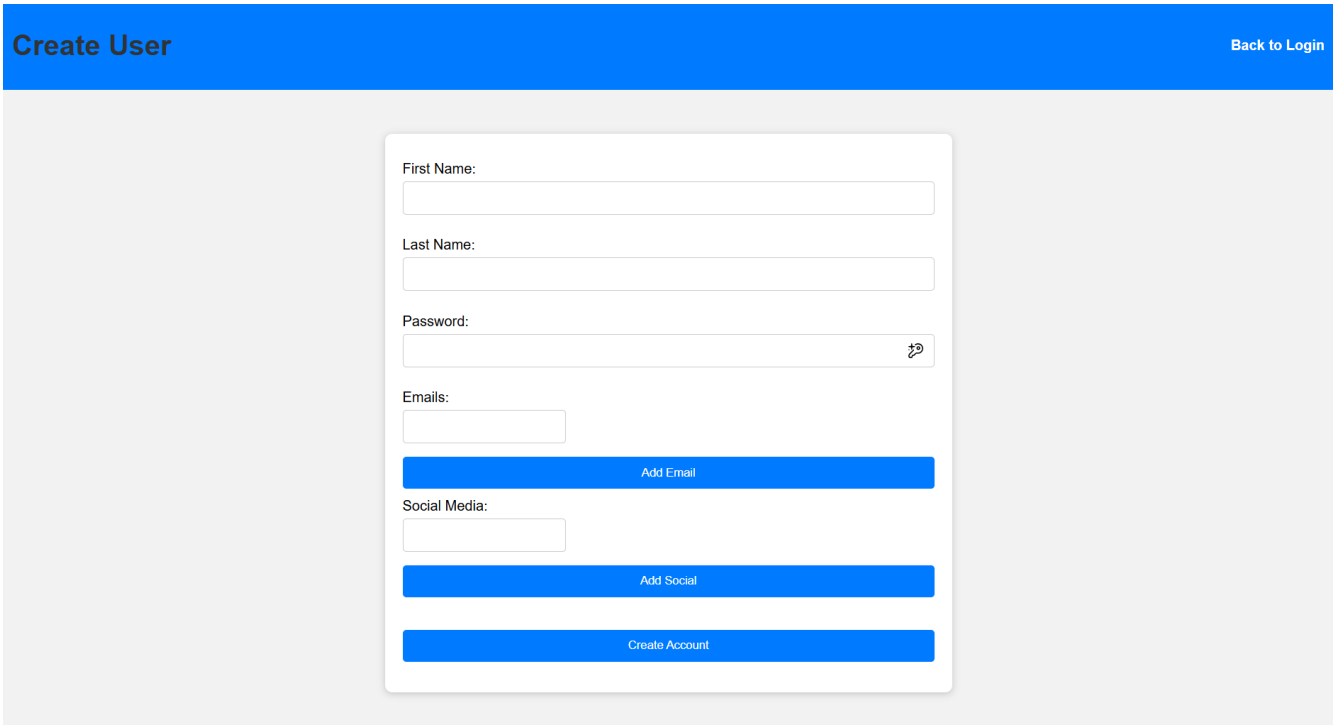
```
    return redirect(url_for('login'))


# ============================================================

# RUN APP

# ============================================================

if __name__ == "__main__":

app.run(debug=True)
```

It does allow creation of a new user:



and all of the html pages load into the local host after troubleshooting.


127.0.0.1:5000
Login
Username:
Password:
Login
Don't have an account? Create User


Create User
Back to Login
First Name:
Last Name:
Password:
Emails:
Add Email
Social Media:
Add Social
Create Account

Sprint 2 Demo Video

Each team must submit a demo video.
All members must participate.

Demo Video Template:

I don’t really have much to demonstrate. The UI and database need more work.

Demo Video (YouTube – Unlisted): https://youtu.be/U1qFzT2a6oc

GitHub Repository Link: https://github.com/WSU-JB5757/451Project.git

.md report for sprint2 is on GitHub as well.